

Project Work — Financial Mathematics

Benassi Martina

Consiglio Giuseppe

Cortivo Davide

Menarbin Samuele

A.A. 2025/2026

1 Introduction and Architecture

The goal of this project is to extend the pricing library `net.finmath.tree` with two main features: dividend modelling inside the three recombining tree models already available (CRR, Jarrow–Rudd and Boyle), and a new class for pricing Barrier Options.

Financial background. Recombining trees are discrete-time approximations of asset price models that, by their nature, evolve in continuous time. We focus on the three following models:

Cox–Ross–Rubinstein (CRR). The binomial tree sets $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$, and recovers the risk-neutral probability $p = (e^{r\Delta t} - d)/(u - d)$ from the martingale condition $\mathbb{E}^Q[S_{k+1} | S_k] = S_k e^{r\Delta t}$. Backward induction prices the option through

$$V_k = e^{-r\Delta t} (p V_{k+1}^u + (1 - p) V_{k+1}^d), \quad V_N = \text{payoff}(S_N).$$

Jarrow–Rudd (JR). The Jarrow–Rudd model is a binomial tree that takes the opposite calibration approach with respect to CRR: instead of deriving the transition probabilities from the martingale condition while keeping u and d fixed, it fixes the probabilities directly and lets the up/down factors absorb the whole risk-neutral dynamics. The risk-neutral probabilities are set to $p_u = p_d = \frac{1}{2}$, as in a coin toss; the up and down factors then incorporate the drift:

$$u = \exp\left[\left(r - \frac{1}{2}\sigma^2\right) \Delta t + \sigma\sqrt{\Delta t}\right], \quad d = \exp\left[\left(r - \frac{1}{2}\sigma^2\right) \Delta t - \sigma\sqrt{\Delta t}\right].$$

This forces u and d to be recomputed every time the drift changes.

Boyle trinomial model. Three branches per node ($u = e^{\sigma\sqrt{2\Delta t}}$, $m = 1$, $d = 1/u$) introduce one extra degree of freedom. The probabilities p_u, p_m, p_d are obtained by solving a 2×2 system that imposes the mean $\mathbb{E}^Q[S_{k+1} | S_k] = S_k e^{r\Delta t}$ and the second moment $\mathbb{E}^Q[S_{k+1}^2 | S_k] = S_k^2 e^{2r\Delta t + \sigma^2\Delta t}$. The extra branch generally leads to faster convergence than the binomial alternatives. Since all three models approximate the same Black–Scholes diffusion under Q , namely

$$dS_t = r S_t dt + \sigma S_t dW_t,$$

where r is the risk-free rate and σ the log-volatility, their prices must converge to a common limit as $N \rightarrow \infty$ — a property checked in Exercise 3. Multiplicative (discrete) dividends reduce the stock price by a factor $(1 - \delta)$ at every ex-dividend date without affecting the risk-neutral probabilities. Continuous dividends, on the other hand, are accumulated in time and break this property, so the probabilities have to be revised. Barrier options are exotic derivatives with a continuous monitoring condition: the option is activated (knock-in) or deactivated (knock-out) as soon as S_t crosses a barrier level H .

The whole architecture, both for dividends and for barriers, follows two guiding principles: **backward compatibility** (the original constructors are left untouched) and **Dependency Injection**, which gives the code more flexibility, also for future extensions.

Dependency Injection. A relevant aspect of the project concerns how dependencies between classes are handled. Rather than instantiating their collaborators internally, each tree model receives a concrete implementation of the required interface type through its constructor, delegating the choice of behaviour to runtime. This approach, known as *Dependency Injection*, separates the pricing logic from the dividend model in use, making the system flexible and easy to extend, as shown in the example below:

```
MultiplicativeDividendModel proportionalDiv = new ProportionalDividend(
    proportionalDividendTime, delta);
CoxRossRubinsteinModel crrProportional = new CoxRossRubinsteinModel(spot, rate,
    vol, maturity, steps, proportionalDiv);
```

Exercise 1 — Dividend Classes

For long-dated options, ignoring dividends introduces a systematic error in prices: the model overestimates the value of the underlying and, as a consequence, the value of the options themselves. The aim of this exercise is therefore to provide the library with a flexible representation of dividends, able to cover different distribution models. The hierarchy (package `...dividends`) starts from the deliberately empty interface `DividendModel`, which captures the pure abstract concept of a dividend. It is extended by `DividendEvent` (which forces the implementation of `double[] getTime()`) and by the key interface `MultiplicativeDividendModel`, which introduces the two main methods used to compute the dividend factor:

```
double getCumulativeDividendFactor(double time);           // D(t)
double getForwardDividendFactor(double time1, double time2); // D(t2)/D(t1)
```

The three concrete implementations are:

- **DiscreteProportionalDividend:** handles an array of ex-dividend dates and the parameter $\delta \in [0, 1]$. The cumulative factor $D(t) = \prod_{t_k \leq t} (1 - \delta_k)$ is computed iterating over the dates in a loop. Ex-dividend dates that do not coincide with a node of the time grid are mapped automatically to the next available node.
- **ProportionalDividend:** the limiting case with a single ex-dividend date, it extends `DiscreteProportionalDividend` by calling `super(delta, new double[]{time})`, which avoids code duplication.
- **ContinuousDividendYield:** models a percentage yield q paid continuously, with the closed-form expressions $D(t) = e^{-qt}$ and $D(t_1, t_2) = e^{-q(t_2 - t_1)}$. The method `getTime()` throws an `UnsupportedOperationException` since the ex-dividend dates would be infinite; `getDividendYield()` is exposed to the tree models in order to adjust the risk-neutral drift when computing the transition probabilities.

Exercise 2 — Tree Models with Dividends

The class `AbstractRecombiningTreeModel` has been equipped with a third constructor that takes a `MultiplicativeDividendModel` and initialises `timeDiscretization[]` (with $t_i = i \cdot \Delta t$). The field is `protected` to allow subclasses to access it while keeping the abstract class independent of the concrete dividend type. Each of the three models (CRR, JR, Boyle) exposes a constructor that takes the dividend and forwards it via `super(...)`, ensuring backward compatibility: when `dividendModel == null`, the `dividendFactor` stays equal to 1 and the behaviour is identical to the original library.

Discrete proportional dividends only act on node values: the transition probabilities remain unchanged, since $e^{r\Delta t}$ is not affected. The continuous dividend yield, on the other hand, modifies

the risk-neutral drift: $e^{r\Delta t} \rightarrow e^{(r-q)\Delta t}$. The check `instanceof ContinuousDividendYield` handles this dichotomy, but with a structural difference between the models: in CRR and Boyle this check happens inside the constructor, since the probabilities are computed once and for all when the object is instantiated; in Jarrow–Rudd it is postponed to the method `buildSpotLevel`, since the factors u and d depend on the drift μ^* and have to be recomputed at every level of the tree. In every model, `buildSpotLevel(k)` initialises `dividendFactor = 1` and updates it conditionally:

```
if (dividendModel != null
    && !(dividendModel instanceof ContinuousDividendYield)) {
    double currentTime = getTime(k);
    dividendFactor = dividendModel.getCumulativeDividendFactor(currentTime);
}
```

In this way, if the model is continuous or if the dividend is absent, the factor remains 1 and does not alter the node values; only in the discrete case is the multiplicative correction $D(t_k) = \prod_{t_i \leq t_k} (1 - \delta_i)$ applied.

Exercise 3 — Impact of Dividends

We built twelve model instances (3 models $\times$ 4 configurations: no dividend, single proportional, discrete, continuous yield) to price European and American Calls and Puts.

Theoretical check. A classical result in option pricing states that, on a non-dividend-paying underlying, early exercise of an American Call is never optimal: the holder would lose the residual time value without receiving any benefit, hence $V_{Am} = V_{Eu}$. Once a sufficiently large dividend is introduced, it becomes rational to exercise early, just before the ex-dividend date, so that $V_{Am} > V_{Eu}$. The automated tests confirm this:

```
Assert.assertTrue(usCRRDiscretePropDiv + 1e-12 >= euCRRDiscretePropDiv);
```

Consistency among models. The three trees approximate the same Black–Scholes diffusion: prices must converge to a common value as $N \rightarrow \infty$. Pairwise consistency was verified within a tolerance of $2 \times \text{tol}$ for every dividend configuration.

Exercise 4 — Barrier Options

The class `BarrierOptionsProducts` extends `AbstractNonPathDependentProduct` and allows the user to price all eight barrier variants (Down/Up $\times$ In/Out $\times$ Call/Put) through the IN–OUT parity, without having to implement a backward induction for each type.

Logic of the class methods. The method `public RandomVariable[] getValues(double evaluationTime, TreeModel model)` calls two private methods: `getValuesViaInOutParity(evaluationTime, model)` when the barrier is of type IN, and `getValuesForOutOption(evaluationTime, model)` when the barrier is of type OUT. These methods in turn proceed recursively until the price at the desired `evaluationTime` is obtained (see Figure 3 for the call chain annotated with the numbered steps).

Extension of the parity to the rebate case. To deal with the presence of a rebate R we built an OUT-barrier without rebate V_{out}^{NR} , whose payoff is defined as the standard out-barrier payoff Π minus the rebate R . Subtracting R produces two key effects: (i) if the barrier is touched, the out-barrier normally pays R , hence $V_{out}^{NR} = R - R = 0$ and the parity gives $V_{in} = V_{Eur}$ — the IN-option activates correctly; (ii) if the barrier is never touched, $V_{out}^{NR} = S_t - K - R$, so the parity gives $V_{in} = V_{Eur} - (S_t - K - R) = R$ — and therefore the payoff of the IN matches the rebate, as expected.

Implementation. `getValuesViaInOutParity` instantiates an `EuropeanNonPathDependent` object with the same payoff, then a mirror out-barrier (`DOWN_IN` $\rightarrow$ `DOWN_OUT`, `UP_IN` $\rightarrow$ `UP_OUT`) with `rebate` = 0 and payoff redefined as $s \mapsto \pi(s) - R$, and finally subtracts the two `RandomVariable[]` arrays element by element.

Backward induction in `getValuesForOutOption`. At the terminal node, each node receives the vanilla payoff if the barrier has not been crossed, otherwise the rebate. Going back node by node, the knock-out condition is checked at every step through `isKnockedOut`:

```
if (isKnockedOut(spotArray[i])) { currentLevel[i] = rebate; }
else                               { currentLevel[i] = contArray[i]; }
```

Exercise 5 — Comparison with the Black–Scholes Formulas

The barrier prices computed by the trees are compared with the analytical benchmark provided by the method `BarrierOptions.blackScholesBarrierOptionValue` in the package `it.univr.fima.barrieroptionsformulas`, a class supplied directly in the project template in place of the native `finmath` implementation, which contains a known typo in one of the coefficients of the closed-form formula. The four macro-types of barrier options were instantiated on the three models with a zero dividend factor. The zero-dividend assumption is appropriate for barrier options on Forex underlyings or commodities. We point out, however, that the class `BarrierOptionsProducts` is not limited to this setting: it can accept any `MultiplicativeDividendModel`, should one wish to include dividends.

Results. For each of the four barrier types, the absolute difference between the tree price and the closed-form formula shrinks as the number of steps N grows. The results confirm the convergence of the three models to Black–Scholes.

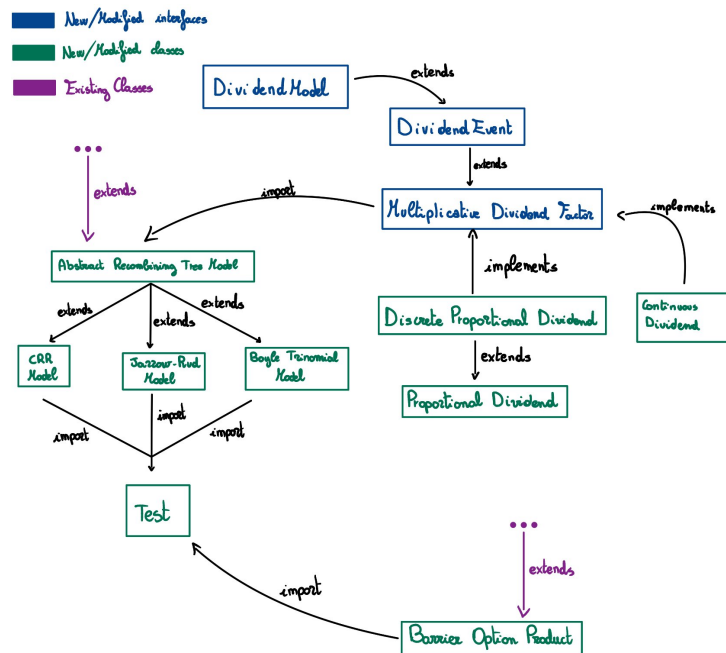
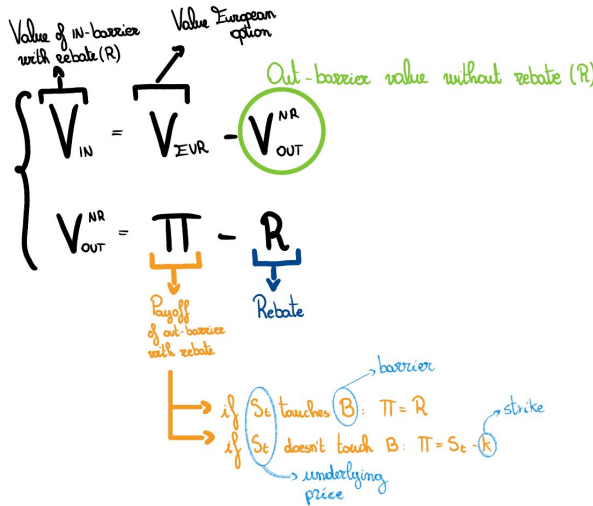


Figure 1: Map of the project class hierarchy: blue = new/modified interfaces, green = new/modified classes, purple = pre-existing classes.



- if B is touched:

$$V_{OUT}^{NR} = R - R = 0$$

$$V_{IN} = V_{EUR} - 0 = V_{EUR}$$

- if B is not touched:

$$V_{OUT}^{NR} = S_t - K - R$$

$$V_{IN} = V_{EUR} - (S_t - K - R)$$

$$= \cancel{S_t} - \cancel{K} - \cancel{S_t} + \cancel{K} + R = R$$

Figure 2: IN-OUT parity modified with rebate: derivation of V_{out}^{NR} and the two cases (barrier touched / not touched).



Figure 3: Call chain for the valuation of barrier options. (1) A `BarrierOptionsProducts` object calls `getValues`. (2) `isInOption()` routes to `getValuesViaInOutParity` for IN barriers. (3) The European option values are computed via `getValues`. (4) The corresponding OUT type is derived (e.g. `DOWN_IN` → `DOWN_OUT`). (5) A new `BarrierOptionsProducts` object of OUT type with `rebate = 0` is created. (6) Its `getValues` is called, which routes to `getValuesForOutOption`. (7) Since `outType` is OUT, `getValues` calls `getValuesForOutOption`. (8) The backward induction returns the array of OUT barrier prices. (9) The `DOWN_OUT` values are subtracted element by element from the European values. (10) The IN-OUT parity returns the desired `DOWN_IN` price.